

Demystifying Neural Networks an Overview for Beginners

 medium.com/@jdseo/demystifying-neural-networks-an-overview-for-beginners-3688a5ded397

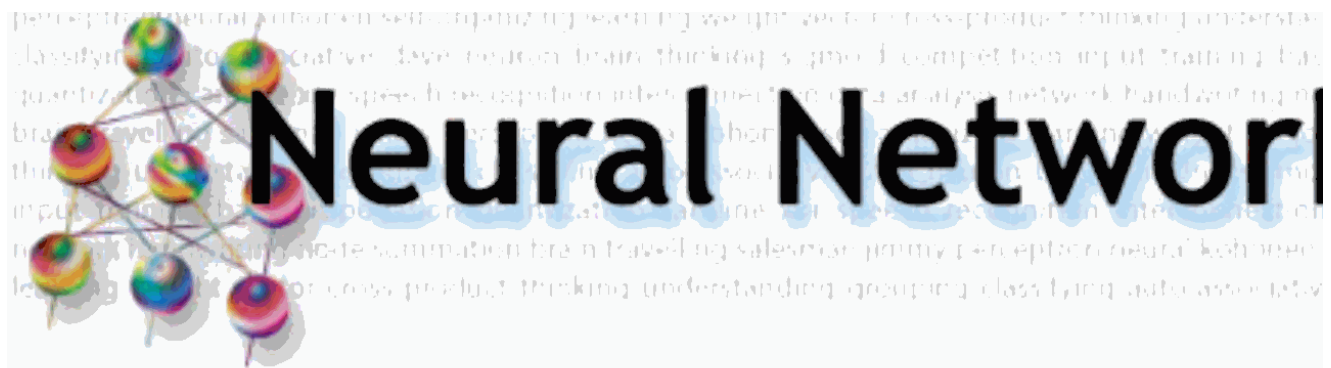
Jae Duk Seo

January 31, 2023

A Journey through the history of Neural Networks to Auto-encoders



Jae Duk Seo



History: The 1940's to the 1970's

In 1943, neurophysiologist Warren McCulloch and mathematician Walter Pitts wrote a paper on how neurons might work. In order to describe how neurons in the brain might work, they modeled a simple neural network using electrical circuits.

gif showing the of Neural Network research

Starting from the paper published back in 1943 by neurophysiologist Warren McCulloch and mathematician Walter Pitts, the research around neural networks started to form shape. First, it was all theory, theory about how neurons work and how they form connections with one another. Slowly people figured a way out to mimic the behaviors of neurons inside a computer program.

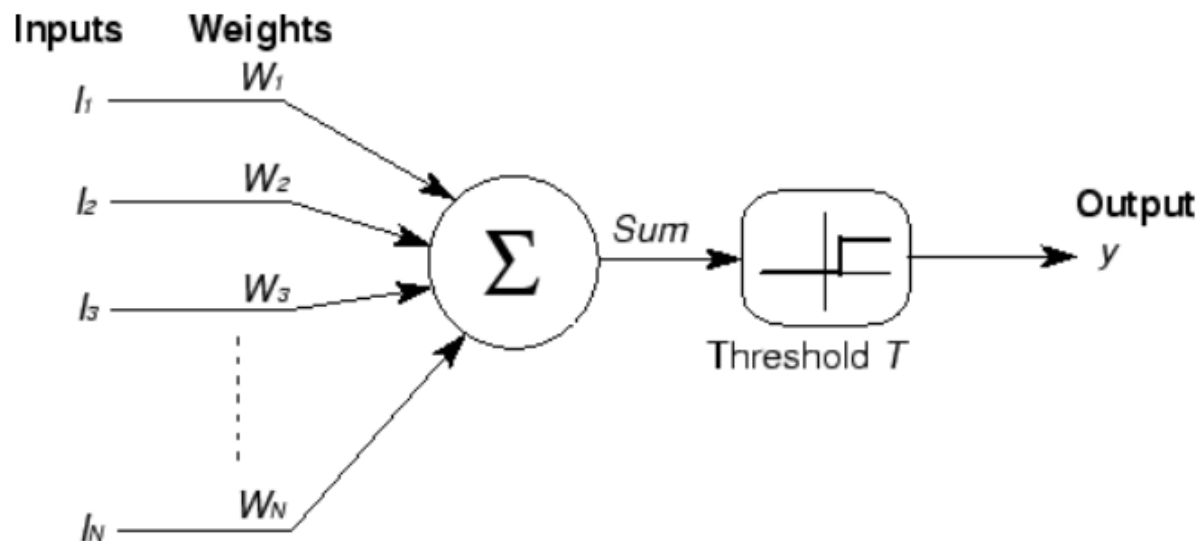


diagram depicting an early of neuron

The first kind of neural network and the simplest was called 'Perceptron'. And what's so fascinating about neural networks is the surprising fact that state-of-the-art AI models are (mostly) nothing but multiple perceptrons all working together. Effectively the difference between then and now is the scale of these networks and the amount of data available for training.

While it was a fruitful effort, early implementations of AI models were just too complex to use and complicated to set up. It took up a lot of space due to hardware sizes and needed a large amount of power. Compared to tools we have now these days such as Chat GPT or Stable Diffusion, it was a VERY specialized and hard-to-use tool.

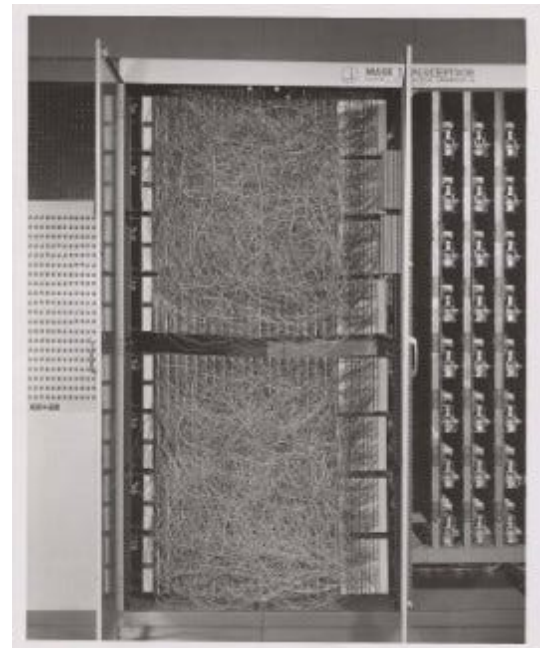
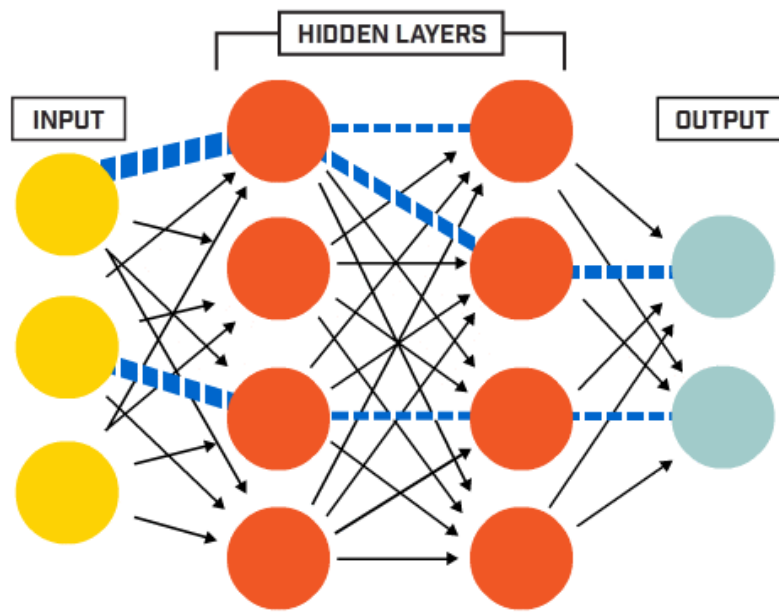


image showing the first of neural networks



gif showing a simplified way of how neural network

Basic Overview

We have all heard the phrase “*Learn through repetition*”, that’s because repetition is how we learn new concepts/ideas/activities as human beings. Biologically speaking, our brain is a collection of neurons all working together to solve problems of life. And different parts of the brain are specialized to do different things.

Neurons Fire 🔥 together Wire 🧠 together

Artificial neural networks (ANN) essentially try to mimic this behavior by using mathematics. In a lot of literature, ANNs are depicted as circles and dots, it’s easy to draw connections between the dots just like how our neurons are wired together. The dots are called and if we have multiple nodes connected on top of one another a forms.

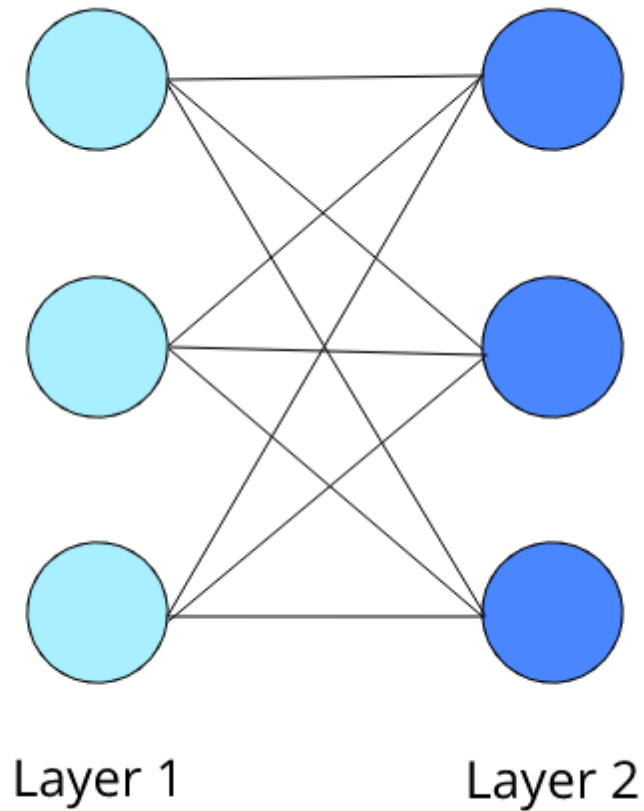


image showing an with 2 layers and 6 nodes

So above visual depicts an artificial neural network with 6 nodes altogether forming a 2-layered artificial neural network. The famous GPT3 model released from OpenAI has 96 layers, and some researchers even attempted to train a . While it is generally accepted by the AI community that more layers usually indicate an AI model with more capacity to learn. There is a point of diminishing returns when it comes to scaling.

Mathematical Overview

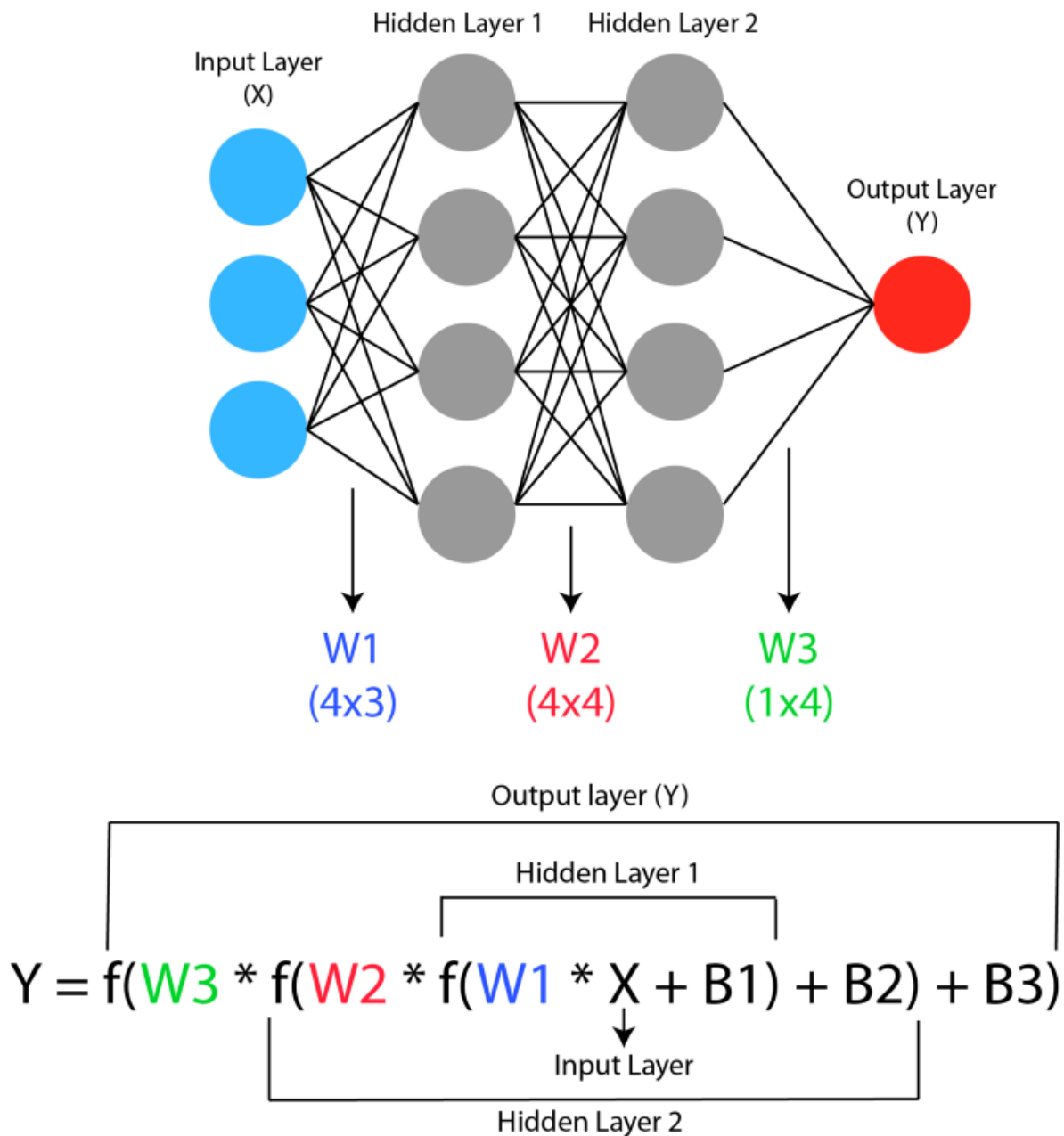
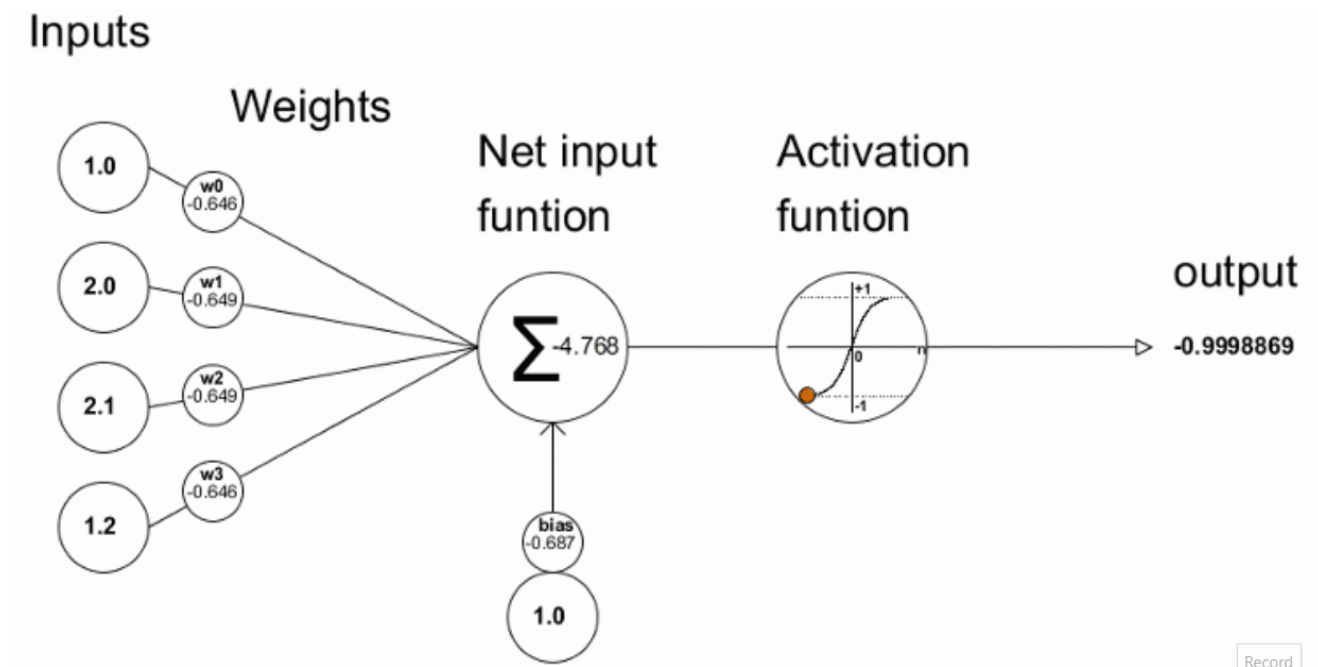


image showing the mathematical representation of 2 layered

So how does all this look like when we actually translate it into mathematics? It would look something like the above, complicated functions being applied again and again. But the color of the weight with the corresponding layer shows how the input data " gets transformed while it travels through the neural network. One of the reasons why it is hard to optimize a neural network with deeper layers is that as the deeper the network gets more calculation is needed. During those calculations, some numerical instabilities could lead to a failure of the whole optimization process. The first layer is called the '*input layer*', the final layer is called the '*output layer*', and all the layers between are called '*hidden layers*'.

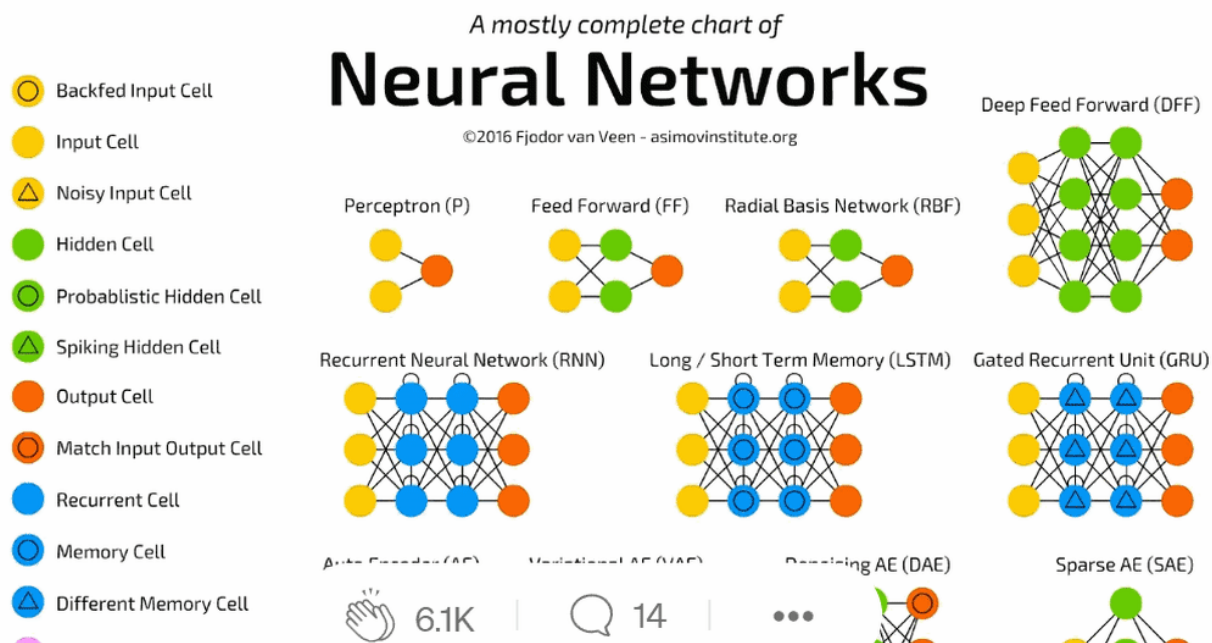


gif showing the inner workings of a mathematical

And inside each of those circles (which are called Perceptrons), a mathematical operation happens that resembles the above visualization. Going back to how our biological neurons function, think of it as this, there is some kind of input data that is coming in. The neuron considers many different sides of this data, these are called weighting.

In the above gif, there are **4 numbers** that are coming into the '*net input function*' (neuron) and **1 number** coming out called the '*output*'. Those four numbers all have different weighting values, which are **w_0** , **w_1** , **w_2** , and **w_3** . After this perceptron has been fully trained, all those w values will have a different number associated with them. Giving different weights to the input data.

Finally, the calculated **numerical value** passes through an activation function, in the above, we can see that the Sigmoid activation function was used. And if the calculated **numerical value** is large enough it will be close to 1 else it will be close to 0. This behavior mimics how our neurons fire at different magnitudes depending on which other neuron it's communicating with.



gif showing different kinds of

Applicational Overview

Just like in Lego, you can create different buildings from the same basic lego blocks, while naming each building differently. We can construct complex networks that have different structures. This concept is called network architecture. And depending on how we shape the network, we can transform data in a different manner. For example, let's take a look at the below visualization.

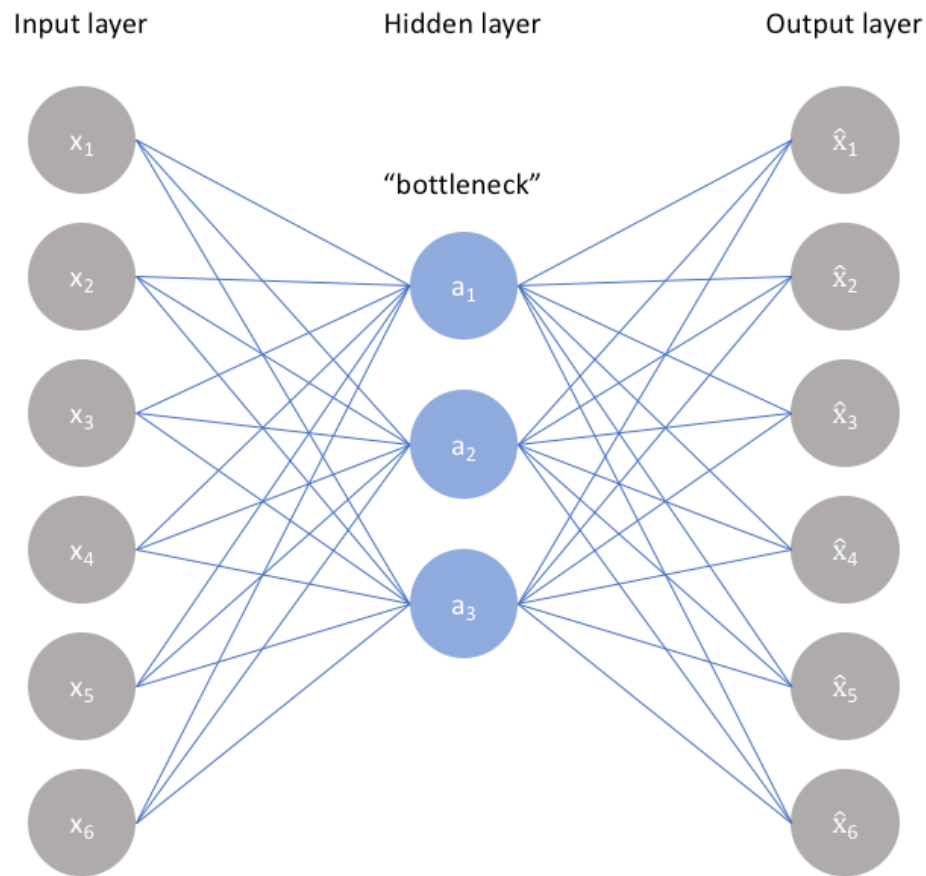


image showing the network architecture of autoencoders

The input layer has as well as the output node (both represented by the grey dot), but the hidden layer only has 3. So while the input data is flowing through the network, it needs to find some kind of a way to compress the input data. Originally, we had 6 ways to represent something, now we need to shrink that number into 3. For example, let's say we have 6 ways to describe a dog, 1) Color, 2) Fur type, 3) Height, 4) Weight, 5) Owner name, and 6) Place of Birth. Now we need to shrink that into 3, what should we do?

| Auto-encoders are good at compressing data 

Compression, in this case, would be learning only the important parts, or finding a way to represent the same thing with fewer words. This might lose accuracy but we will gain some kind of generality among the attributes that we just discussed. We can group 1) and 2) into Dog Breed, group 3) and 4) into Dog Dimension, and groups 4) and 5) into Dog Owner Information. This is not the most accurate example, but the idea is simple, auto-encoders are good at finding new ways to encode data. Important to note that to make all of this work, the auto-encoder needs to be trained to reconstruct the original data.

Autoencoders

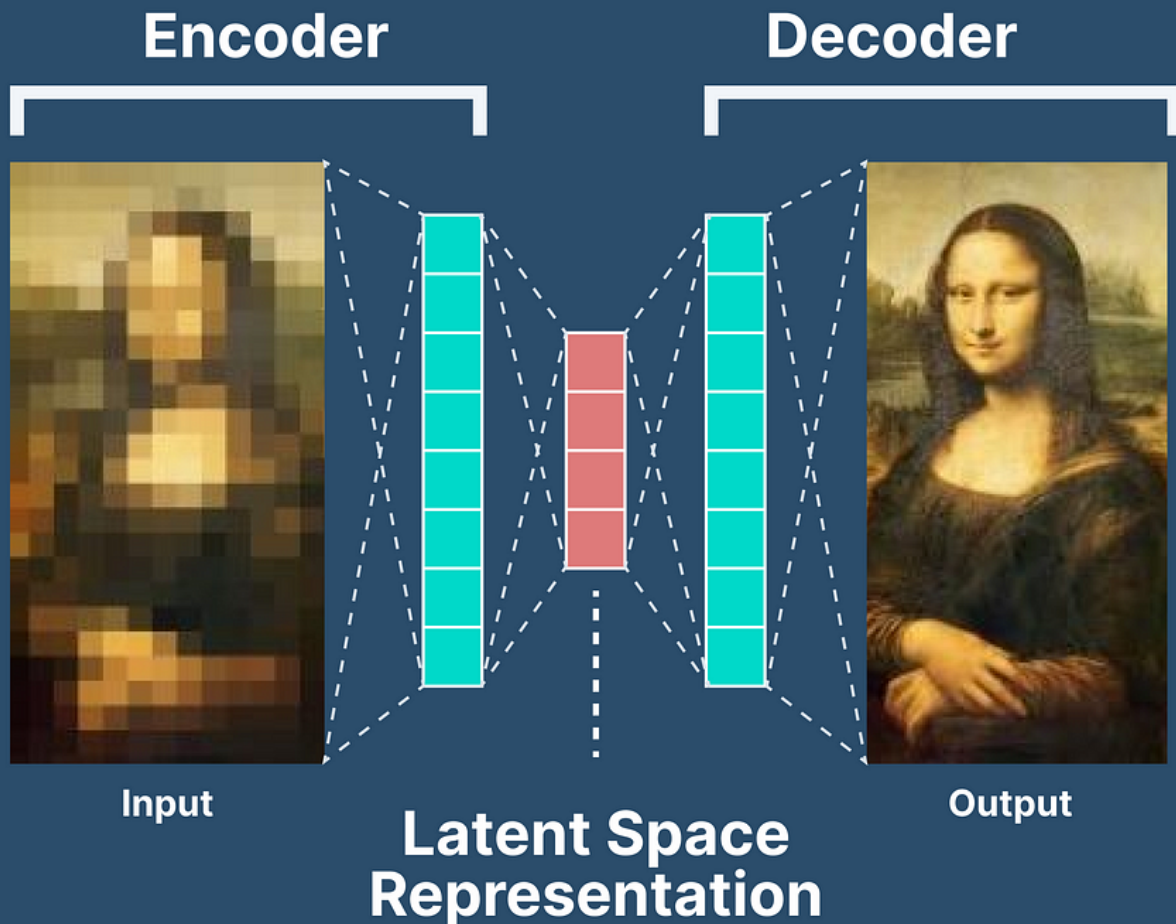


image showing the usage of auto-encoders

And learning how to effectively compress data have a lot of useful properties. If we follow the step-by-step procedure of what the AI model is doing first, it is given the input data and needs to find a way to represent that same data with fewer attributes. After this, it needs to find a way to reconstruct the original data from the compressed version. It can only do this operation successfully if it learns how to extract only the fundamental attributes to describe the data. Those learned attributes can be applied to other incomplete data to recover the

original, leading to applications like those shown above where a pixelated image gets to be converted into a high-resolution one. This operation is actually called and there is even a whole study around this topic.

Thank you for reading, now you know perceptron, you might be wondering what about Chat GPT? What kind of business use cases does it have? Then I recommend you my article “”.